Foundation of Blockchain Technology (IS501)
Pre-Reading Material – Experiment 2
Deploying and Interacting with Smart Contracts Using Remix IDE

Estimated Reading Time: 20–25 minutes

Reference Reading

Ethereum Blockchain Developer — Remix IDE: Starting, Stopping and Interacting with Smart Contracts

Experiment Context

In this experiment, you will work with a **smart contract using Remix IDE**, a browser-based development environment for Ethereum smart contracts.

The experiment focuses on understanding the basic workflow of a smart contract:

Write Contract → Compile → Deploy → Start/Stop → Interact → Observe Results

The objective is to understand how a Solidity smart contract is deployed and how its functions can be accessed through Remix.

Learning Outcomes

After completing this experiment, you should be able to:

- Identify the main components of the **Remix IDE**.
- Understand the basic structure of a Solidity smart contract.
- Compile a smart contract using Remix.
- Deploy a smart contract.
- Understand the difference between **deploying** and **interacting** with a contract.
- Call functions provided by a deployed smart contract.
- Understand how transactions and contract state changes are reflected in Remix.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Activate Your Prior Knowledge

Consider the following question:

What happens after a Solidity smart contract is written?

Writing the code is only the first step.

A typical smart contract development process is:

Write → Compile → Deploy → Interact

Think about:

- Where is the contract compiled?
- Where is the compiled contract deployed?
- How can a user call its functions?

Prerequisite Concepts

Before performing this experiment, you should have a basic understanding of:

- Blockchain
- Ethereum
- Smart contracts
- Solidity
- Transactions
- Wallet/accounts
- Gas
- Ethereum addresses

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Core Concepts

1. Remix IDE

Remix IDE is an online development environment commonly used for writing, compiling, deploying, and interacting with Ethereum smart contracts.

It provides tools for several stages of smart contract development, including:

Code Editor → Compiler → Deployment → Contract Interaction

For this experiment, Remix acts as the primary interface through which you will work with the smart contract.

2. Smart Contract

A **smart contract** is a program stored and executed on a blockchain.

A simple Solidity contract may contain:

- Variables
- Functions
- State-changing operations
- Read-only operations

For example, a contract may store a value and provide functions to:

- Set the value
- Read the value

The important idea is:

A deployed smart contract has an address, and users interact with it through its functions.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

3. Compile the Smart Contract

Before deployment, the Solidity source code must be **compiled**.

The compiler checks the Solidity program and generates the information required for deployment and interaction.

The compilation process produces important artifacts such as:

- **Bytecode** — used when deploying the contract.
- **ABI (Application Binary Interface)** — describes how external applications can interact with the contract.

Therefore:

Solidity Code → Compiler → Bytecode + ABI

4. Deploy the Smart Contract

After successful compilation, the contract can be **deployed** using Remix.

Deployment creates a contract instance on the selected blockchain environment.

The deployed contract receives an **address**.

The basic workflow is:

Compile

↓

Select Contract

↓

Choose Environment/Account

↓

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Deploy

↓

Contract Address

After deployment, Remix displays the deployed contract and provides access to its available functions.

5. Interacting with a Smart Contract

Once deployed, users can interact with the contract through its functions.

There are generally two important types of interaction:

Read Operation

A function that only reads information from the blockchain does not modify the contract's state.

For example:

Get stored value

Write Operation

A function that changes the contract's state requires a blockchain transaction.

For example:

Set stored value

This distinction is important:

Read Operation	Write Operation
Reads blockchain state	Changes blockchain state

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Read Operation	Write Operation
Does not modify contract state	Modifies contract state
Generally does not require a transaction	Requires a transaction
Used to retrieve information	Used to update information

6. Starting, Stopping and Interacting

The supplied reference specifically focuses on **starting, stopping, and interacting with smart contracts** through Remix.

Before performing the experiment, observe that a deployed contract can expose functions that allow users to interact with its state.

The experimental workflow can therefore be viewed as:

Deploy Contract

↓

Locate Deployed Contract

↓

Start/Activate Interaction

↓

Call Contract Functions

↓

Observe Output/State

↓

Stop/Terminate as required

The exact controls and interface depend on the Remix environment and the smart contract being used.

Guided Reading Questions

1. What is Remix IDE?
2. Why does a Solidity smart contract need to be compiled?
3. What is the purpose of the ABI?
4. What happens when a smart contract is deployed?
5. What is the difference between a read operation and a write operation?
6. Why does changing the state of a smart contract require a transaction?
7. What information can be observed after deploying a contract in Remix?

Reflection Activity

Imagine that a smart contract stores a student's marks.

It provides two functions:

- **getMarks()**
- **setMarks()**

Think:

Which function would only read information?

Which function would modify the blockchain state?

Would both operations require the same type of interaction?

Write **100–150 words** explaining your answer.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

Self-Assessment

Fill in the blanks

1. _____ is an online development environment used for Ethereum smart contracts.
2. Solidity code must be _____ before it can be deployed.
3. The _____ describes how external applications can interact with a smart contract.
4. A deployed smart contract has a unique _____.
5. A function that changes contract state generally requires a blockchain _____.

True or False

6. A smart contract can be deployed directly without compilation. _____
7. Remix can be used to deploy and interact with smart contracts. _____
8. A read operation changes the state of the smart contract. _____
9. A write operation can modify the state stored by a smart contract. _____

Think Before the Experiment

Before starting the practical, answer:

If a smart contract is already deployed on a blockchain, why can't we simply edit its Solidity source code to change its behavior?

Be prepared to discuss your answer during the experiment.

Key Takeaways

- **Remix IDE** provides an environment for developing and interacting with smart contracts.
- A Solidity contract must first be **compiled** before deployment.
- Compilation produces information such as **bytecode and ABI**.
- Deployment creates a contract instance on the selected blockchain environment.
- A deployed contract has an **address**.

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in

- **Read operations** retrieve information without changing contract state.
- **Write operations** can change contract state and require a transaction.
- Remix provides an interface through which users can interact with deployed contract functions.

Remember

Write → Compile → Deploy → Interact → Observe

Campus : Prestige Vihar, Scheme No. 74-C, Sector D, Vijay Nagar, Indore -452010 (M.P.) India
Phone : 0731-4013333, Fax : 0731-4013329 E-mail : info@piemr.edu.in, Web. www.piemr.edu.in